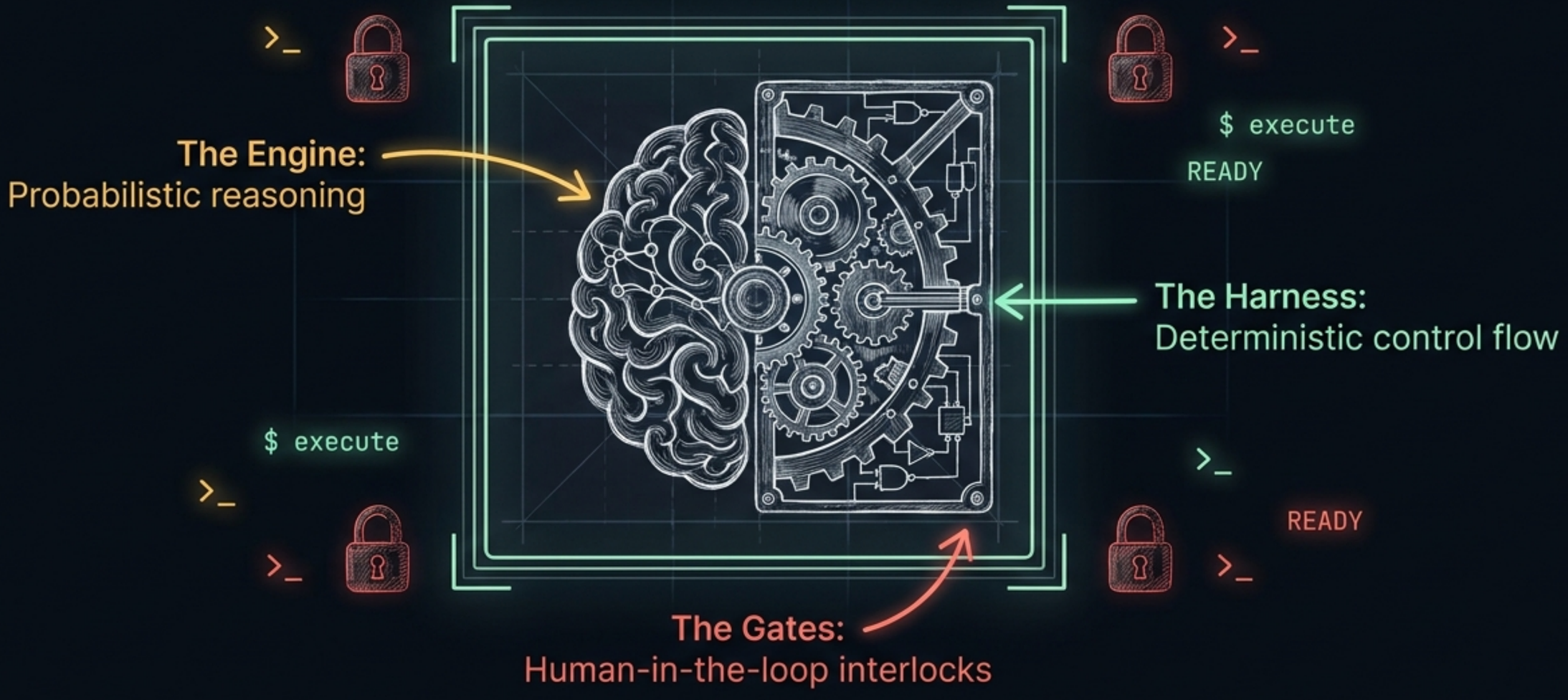
Agentic Design Patterns

An architectural playbook for building enterprise-grade AI systems.



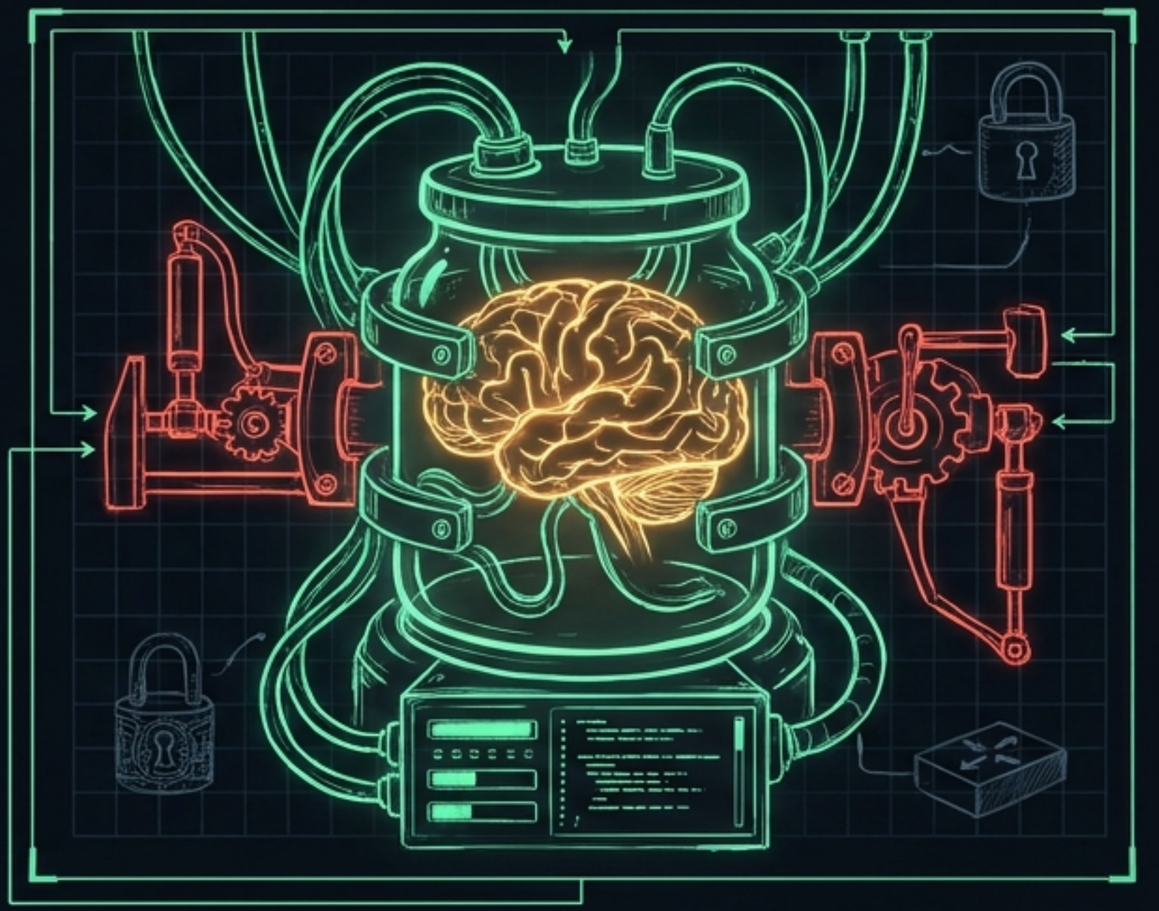
The core is probabilistic. The harness must be deterministic.

A raw language model is a stateless, probabilistic token predictor. It proposes actions but cannot execute tools, enforce invariants, or halt a runaway loop.



The Model (Reasoning Engine)

Dynamic intent parsing, unstructured synthesis, fuzzy matching, multi-step planning.



The Architecture (Deterministic Harness)

Control flow ownership, code-level execution gates, state persistence, strict bounding constraints.

The Simplicity Ladder: Earning the right to autonomy.

Rule 0: Start with the simplest architecture that passes your evaluation suite.
Climb to the next tier only when empirical evaluations prove the simpler tier fails.



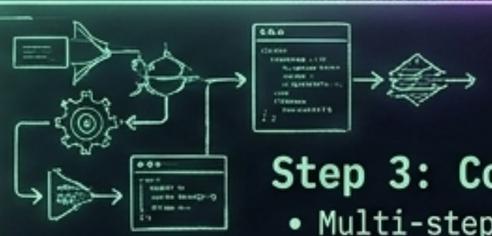
Step 1: Raw LLM

- Stateless token predictor.
- Control Owner: Caller (1 turn).
- Memory: Brain in a jar.



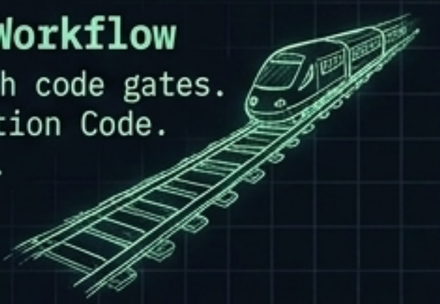
Step 2: Augmented LLM

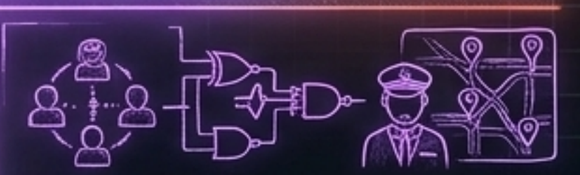
- Single call + Retrieval (RAG) + Tools.
- Control Owner: Caller (1 turn + tools).
- Memory: LLM with a backpack.



Step 3: Composable Workflow

- Multi-step pipeline with code gates.
- Control Owner: Application Code.
- Memory: Railway tracks.





Step 4: Autonomous Agent

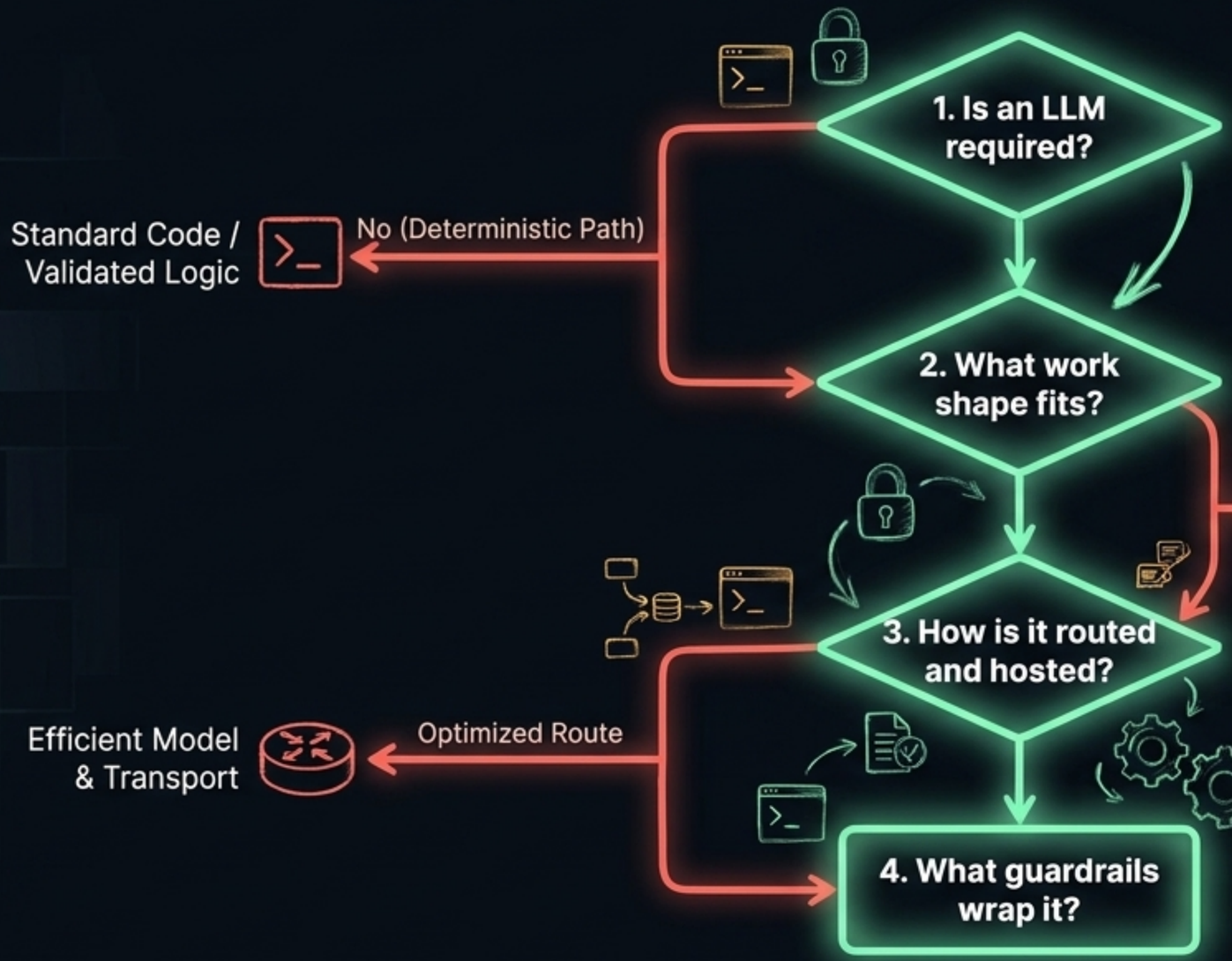
- LLM directs its own multi-turn process.
- Control Owner: The Model.
- Memory: Taxi driver.



Warning:
Unpredictable
trajectory

The 4-Decision Sequence for architectural triage.

Skipping Step 1 or 2 is the primary cause of over-engineered, brittle, or unsafe agentic deployments.



Separate exact math/logic (Deterministic Code) from fuzzy synthesis (LLM) and high-stakes accountability (Human).

```
IF exact_logic OR validation THEN use_deterministic()
ELSE use_llm()
```

Select the lowest necessary rung on the complexity ladder (Single Call -> Workflow -> Agent).

```
complexity = MIN(SingleCall, Workflow, Agent)
```

Match the step to the right model tier (Planner vs. Executor) and transport mode (Batch vs. Stream).

```
route_to(ModelTier, TransportMode)
```

Surround every probabilistic node with pre-checks, schema validators, and independent critique.

```
GUARDED_CALL = PreCheck() -> LLM() -> SchemaValidate() -> Critique()
```

Final Architecture: Guarded, Optimized, and Right-Sized System

Isolating probabilistic work prevents Deterministic Drift.

Never assign Bucket 1 or Bucket 3 responsibilities to Bucket 2.

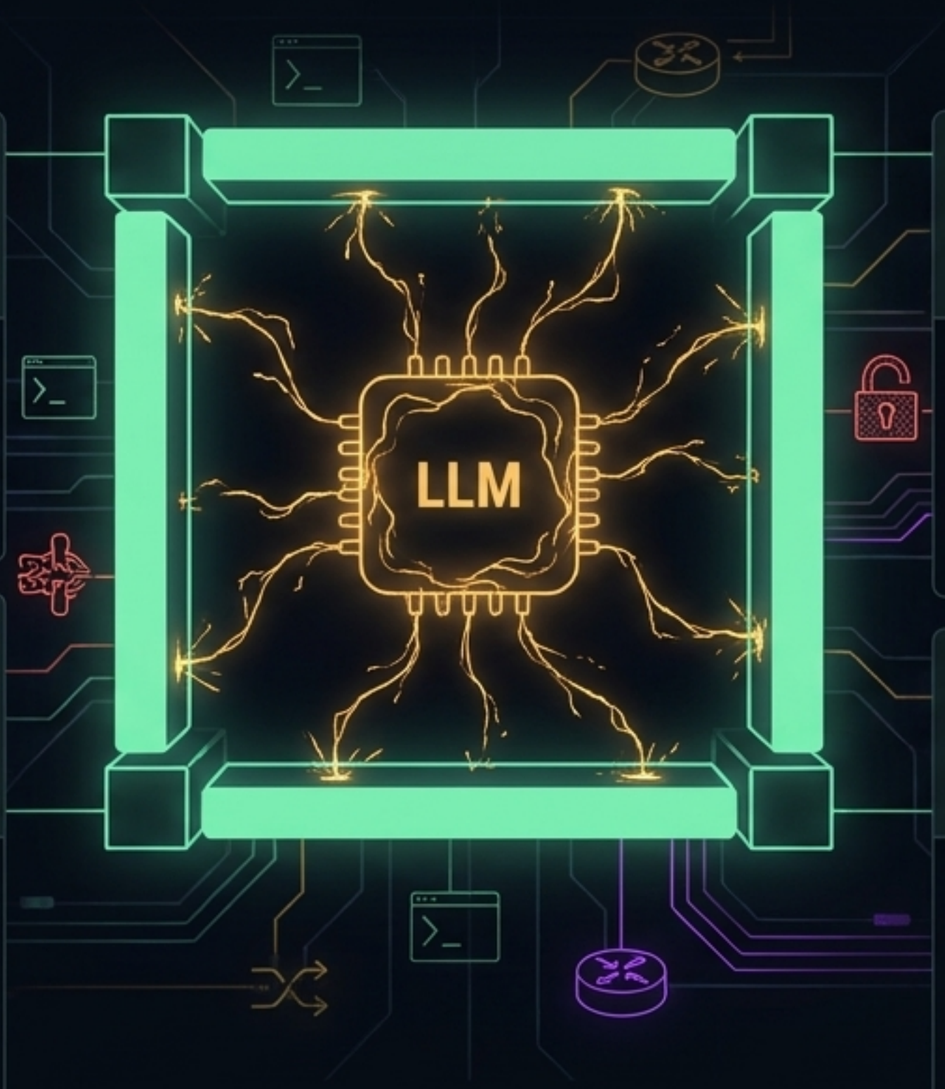


The 5-Factor Pattern Selection Matrix

	Single Call	Workflow	Agent
1. Task Predictability	Fully known, 1 step	Known decomposition, K steps	Unknown/emergent, N steps
2. Latency Budget	Sub-second to 3s	3s to 15s	Seconds to minutes+
3. Blast Radius	Low / Read-only	Medium / Gated	High if ungated
4. Token Cost Ceiling	~1x baseline	~3-5x baseline	~10-15x+ baseline
5. Auditability	High, trivial log	Highest, strict pass/fail gates	Complex, requires full event-ledger tracing

Defending against inherent LLM physics.

Every agentic architecture must explicitly defend against four probabilistic properties.



1. Non-determinism

Identical prompts yield floating-point variance.

Mitigation: Grade via structured schemas at the token level; never test for exact string equality.



2. Plausible-Sounding Errors (Hallucinations)

Models invent confident facts over garbled input.

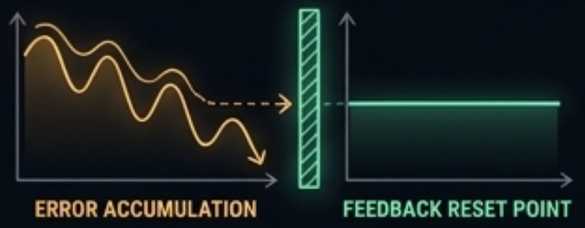
Mitigation: Deterministic pre-LLM epistemic quality gates and strict citation verification.



4. Compounding Multi-step Errors

A 95% reliable step yields a 60% reliable 10-step chain.

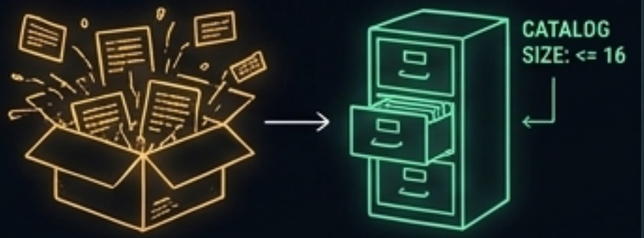
Mitigation: Insert ground-truth environmental feedback after every step to reset the error curve.



3. Context Sensitivity

Packing 30 rules degrades mid-window recall.

Mitigation: Split generation from verification; keep tool catalogs <= 16.



The 5 Canonical Workflow Patterns

When a single call falls short, but the overall control structure is predictable, application code must own the control flow.

1. Prompt Chaining



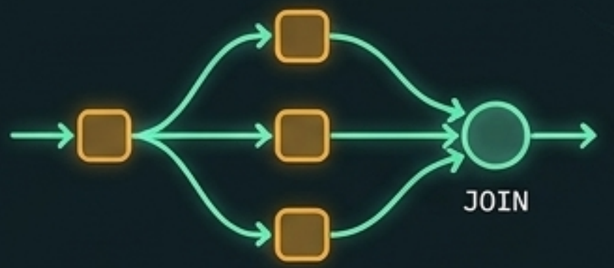
Assembly line

2. Routing



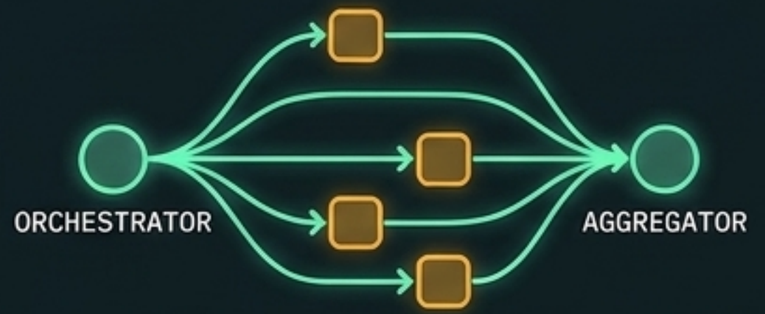
Train tracks

3. Parallelization



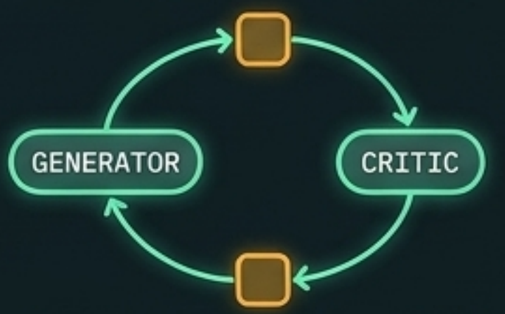
Pizza slices / Polling

4. Orchestrator-Workers



Tech lead & tickets

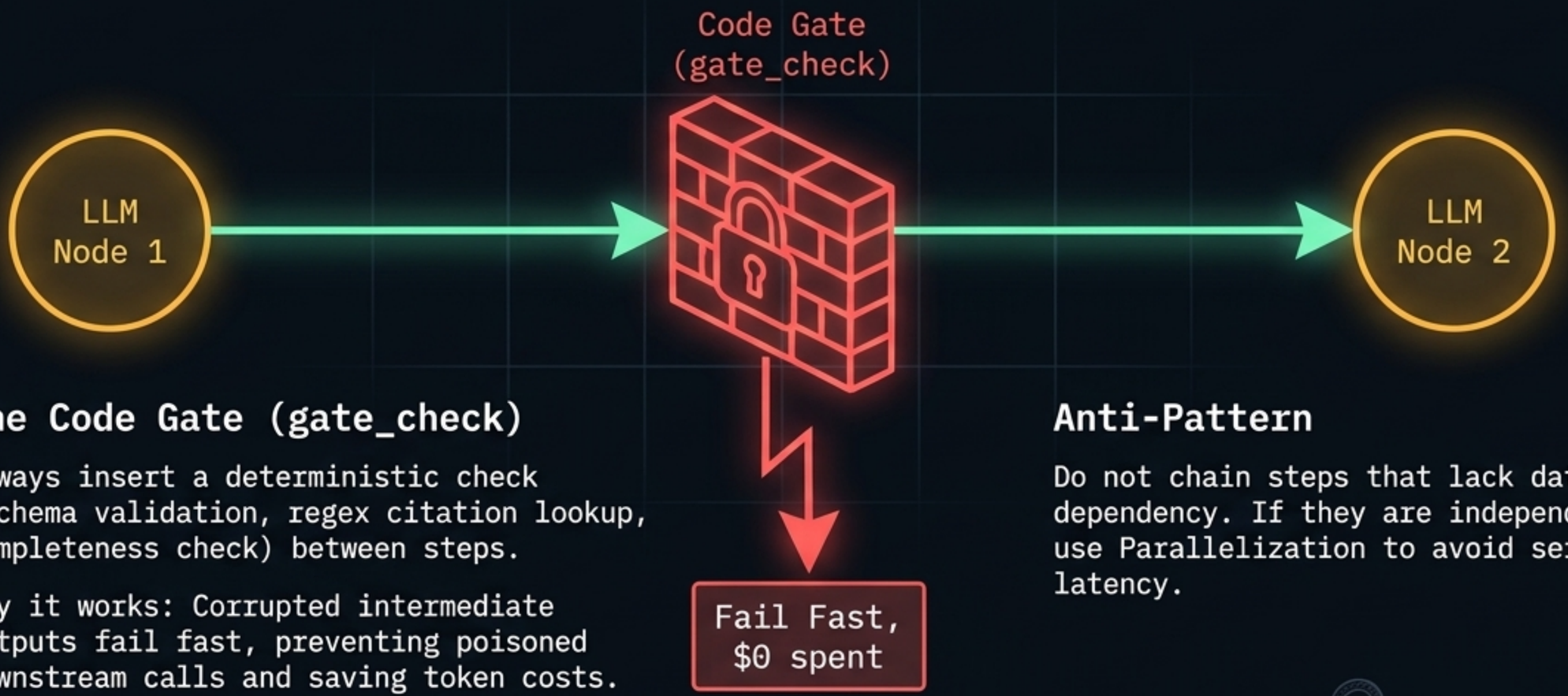
5. Evaluator-Optimizer



Generator & Critic

Pattern 1: Prompt Chaining (Sequential Pipelines)

Decomposes a complex task into a fixed sequence of narrower LLM calls, where each step consumes the validated output of the previous step.



The Code Gate (gate_check)

Always insert a deterministic check (schema validation, regex citation lookup, completeness check) between steps.

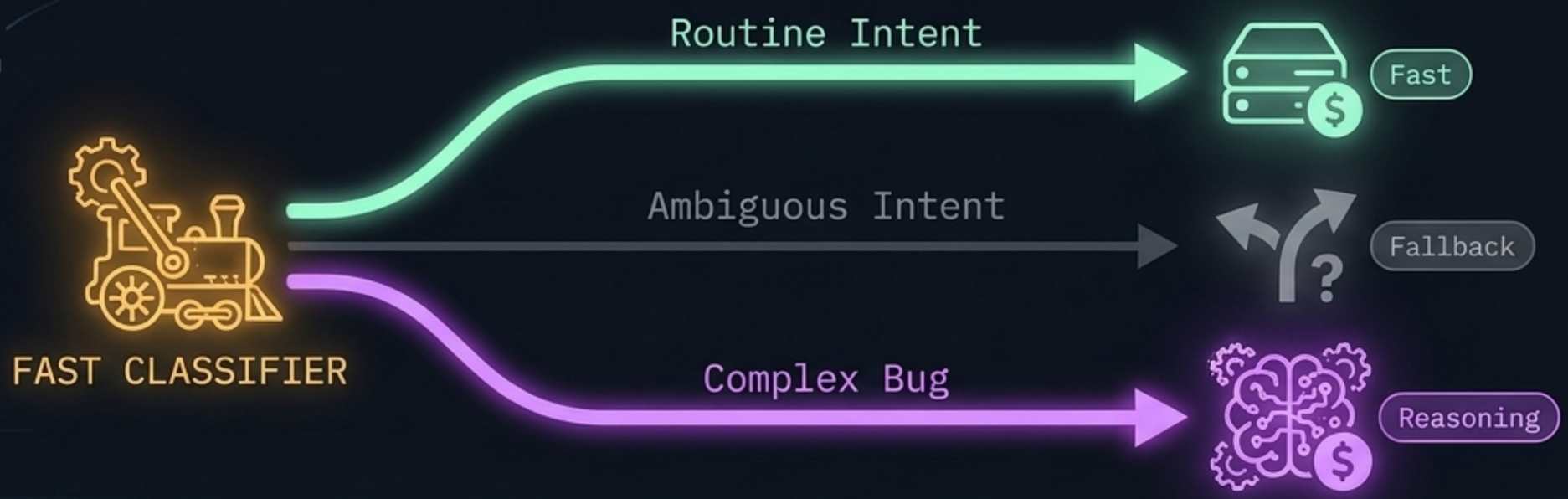
Why it works: Corrupted intermediate outputs fail fast, preventing poisoned downstream calls and saving token costs.

Anti-Pattern

Do not chain steps that lack data dependency. If they are independent, use Parallelization to avoid serial latency.

Pattern 2: Routing (Intent & Tier Dispatch)

Classifies incoming requests and dispatches them to specialized prompts, scoped toolsets, or cost-appropriate model tiers.



Defeating the Monolith

Optimizing one mega-prompt for all edge cases causes instruction clash and forces every call onto the most expensive model tier.

Model-Tier Optimization

Route high-volume, routine intents to fast, low-cost tiers. Reserve frontier tiers for complex troubleshooting.

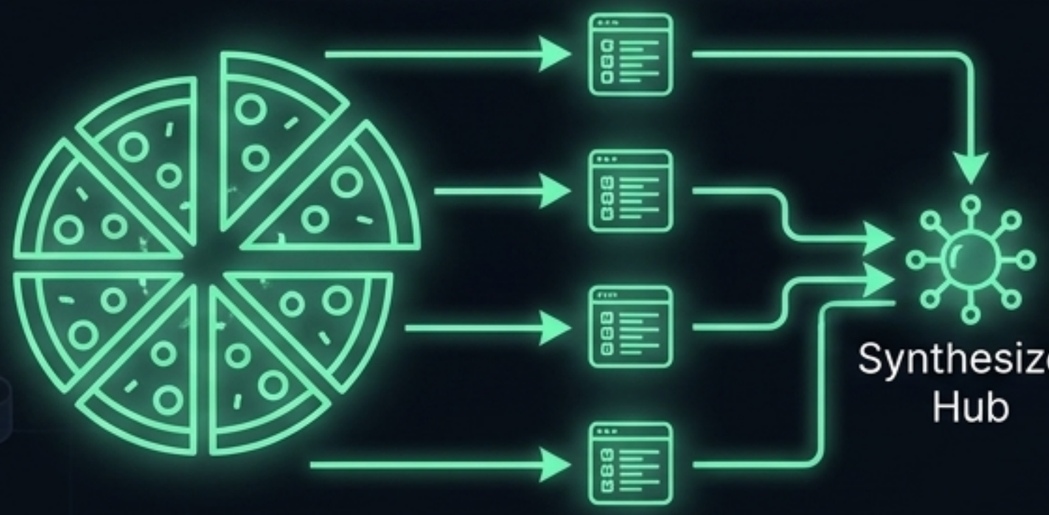
Misrouting Protection

Always include an explicit unclear/ambiguous fallback category rather than forcing a guess.

Pattern 3: Parallelization (Sectioning vs. Voting)

Executes multiple LLM calls concurrently and merges outputs programmatically.

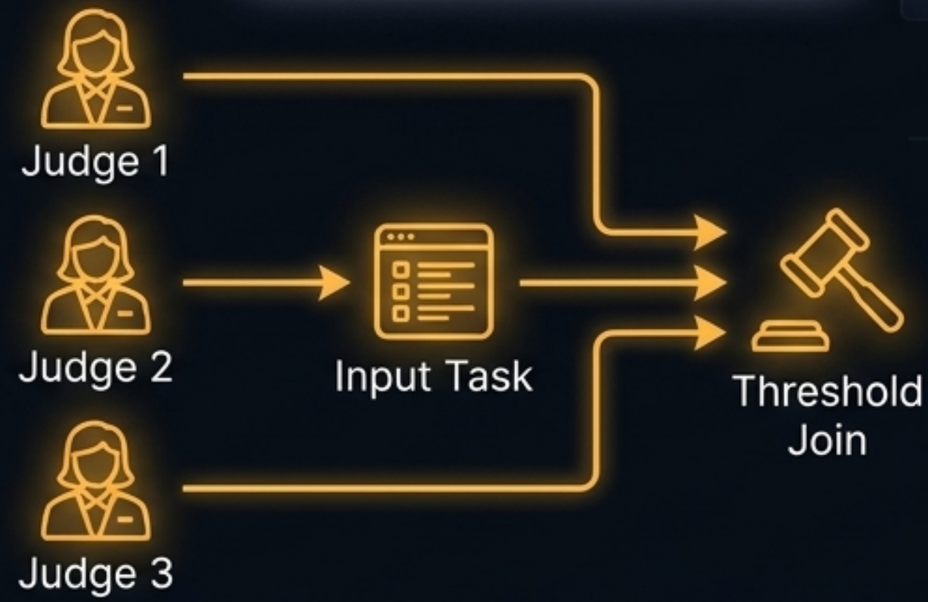
Sectioning (Speed)



Split a task into distinct, independent subtasks. Total latency equals the slowest branch, not the sum of branches.

Rule: Isolate output keys so concurrent workers never overwrite shared state.

Voting (Confidence)

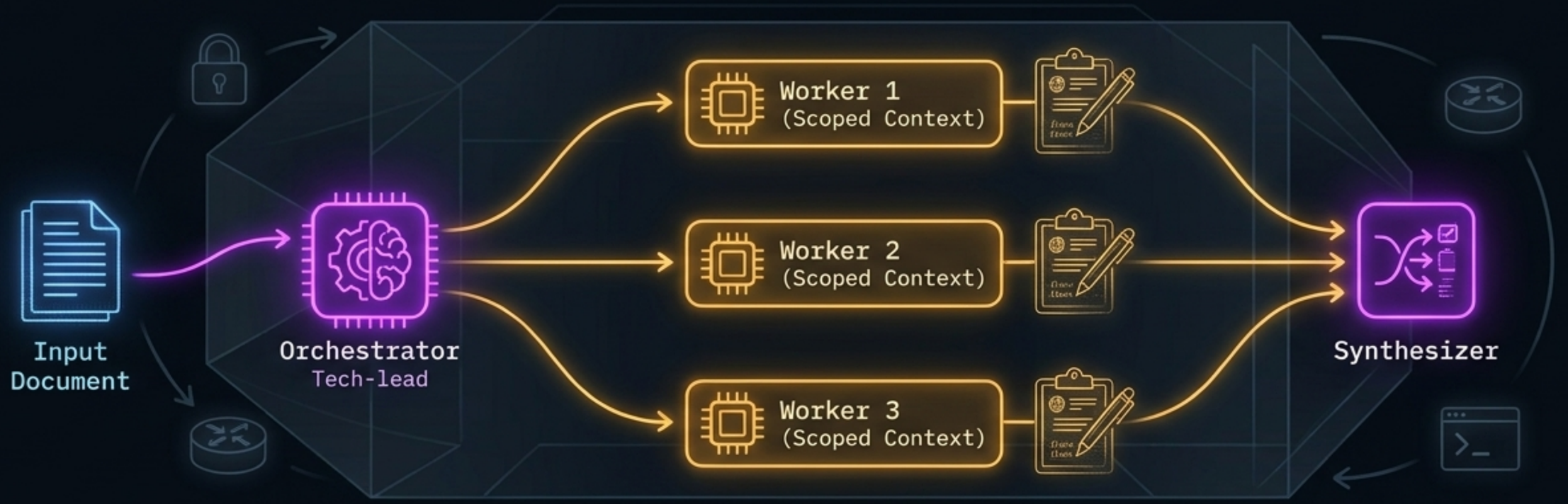


Run the same task multiple times to reduce false negatives on high-stakes reviews.

Rule: Tune threshold to asymmetric risk (e.g., in security scans, flag if any 1 of 3 finds an exploit).

Pattern 4: Orchestrator-Workers (Dynamic Decomposition)

A central model inspects the input, dynamically invents a list of subtasks, dispatches workers with scoped context, and synthesizes the returned summaries.



Dynamic vs. Static

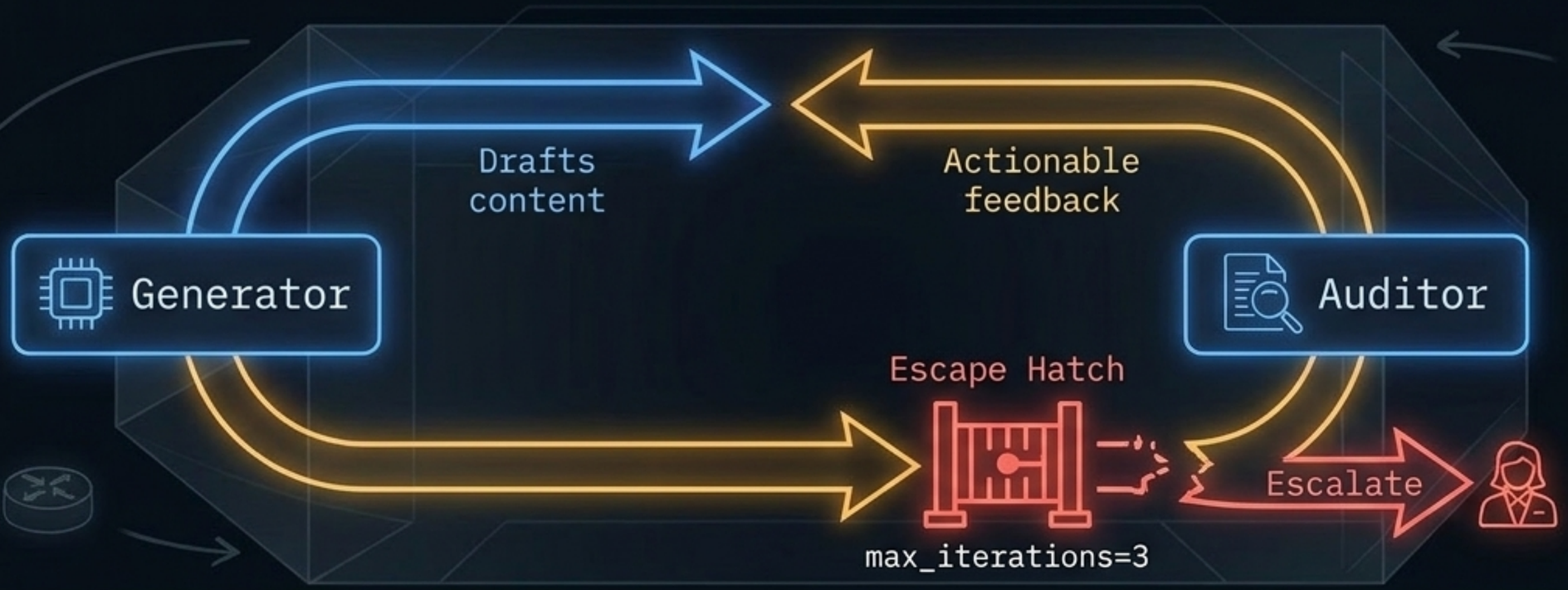
Unlike standard Parallelization (where branches are hardcoded), Orchestrator branches are determined dynamically at runtime based on the specific payload.

Spec-Driven Split

A high-capability model acts as a read-only planner to produce a structured task spec, dispatching mid-tier/fast-tier workers to execute bounded subtasks in fresh context windows.

Pattern 5: Evaluator-Optimizer (Bounded Loops)

Iterative quality refinement pairing a generator with an independent critic, evaluated against a strict rubric.



Separation of Authority

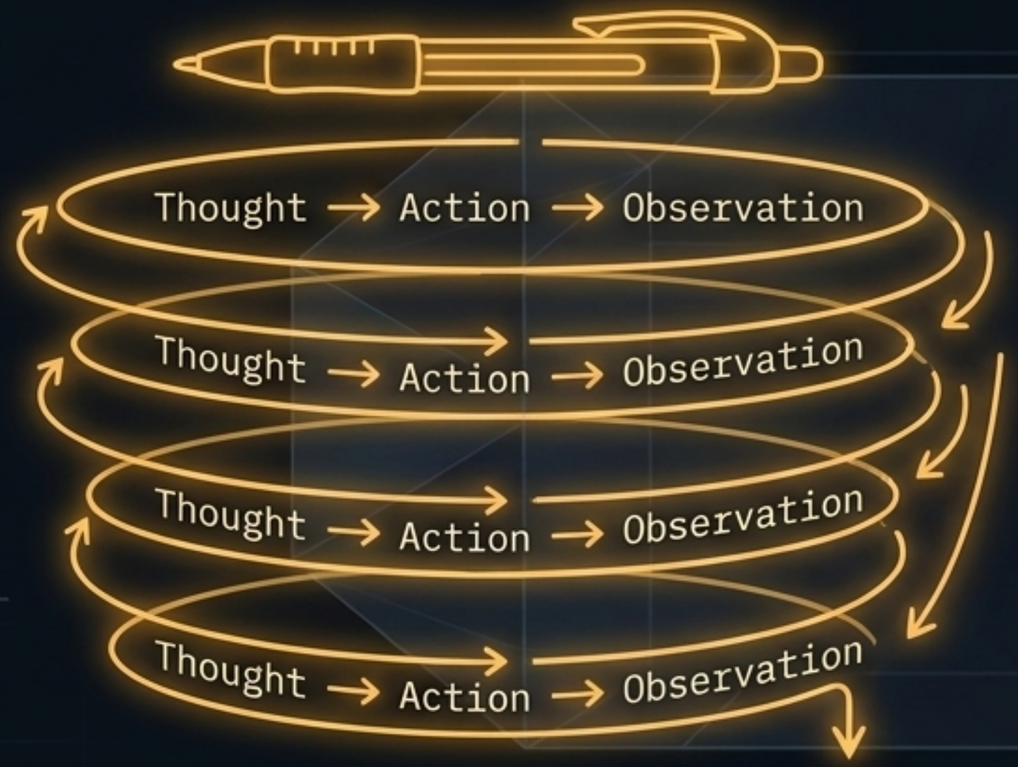
Never ask a model to draft and grade its own work in the same turn—it will rationalize its mistakes. Use a cold-context critic paired with deterministic verification tools.

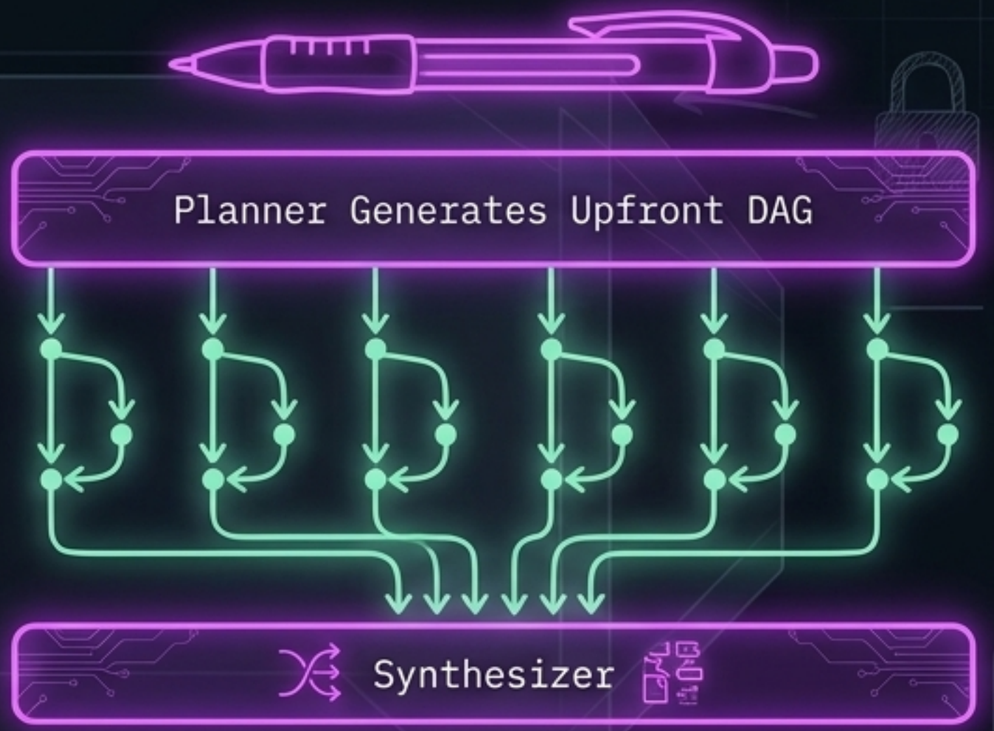
Mandatory Loop Hygiene

Always enforce `max_iterations` and an explicit convergence signal. If the loop trips without passing, never silently ship the unverified draft. Escalate to human review.

Reasoning Loops: ReAct vs. Plan-and-Execute

When step N+1 depends on the discovery in step N, the agent must take the wheel.





ReAct (Reason + Act)

Interleaves thought, action, and observation turn-by-turn.
Strength: Maximum adaptability; grounded in immediate observations.
Risk: High sequential latency (O(N) turns); local rabbit holes.

Plan-and-Execute (ReWOO/LLMCompiler)

Planner generates upfront DAG -> parallel execution -> Synthesizer.
Strength: Slashes latency and token re-ingestion.
Risk: Brittle if early tool observations invalidate the entire upfront plan.

Mandatory Guardrails & Poka-Yoke Interfaces

An autonomous loop without strict termination controls is a production outage waiting to happen.



- Explicit ground-truth exit criteria
- Hard turn & call ceilings (max_turns)
- Cumulative token/cost budgets
- Structured error feedback (is_error: true)
- Duplicate tool-call detection (hash checking)

ACI Poka-Yoke

(mistake-proofing)

Before - Fragile

`path: str`

Swallows exceptions.
Leaks parameters.

After - Structurally Sound

`absolute_path: str`

Strict JSON schema required. Unambiguous format.
Never swallows exceptions as empty strings.

The 4-Tier Memory Architecture

A 1M token context window is not an excuse to dump every file into a single prompt. Memory must be stratified.



1. Working Memory

Active scratchpad (system instructions, live tool observations). Cleared after session.

Optimize for prompt-cache.

2. Episodic Memory

Record of specific past sessions and trajectories for audit and few-shot recall.

3. Semantic Memory

Verified domain knowledge and consolidated user facts (Vector/RAG).

Rule: Consolidate updates rather than appending contradictory logs.

4. Procedural Memory

System heuristics and playbooks (Skills).

Rule: Use progressive disclosure (load names first, full bodies only when triggered).

Defeating Context Rot

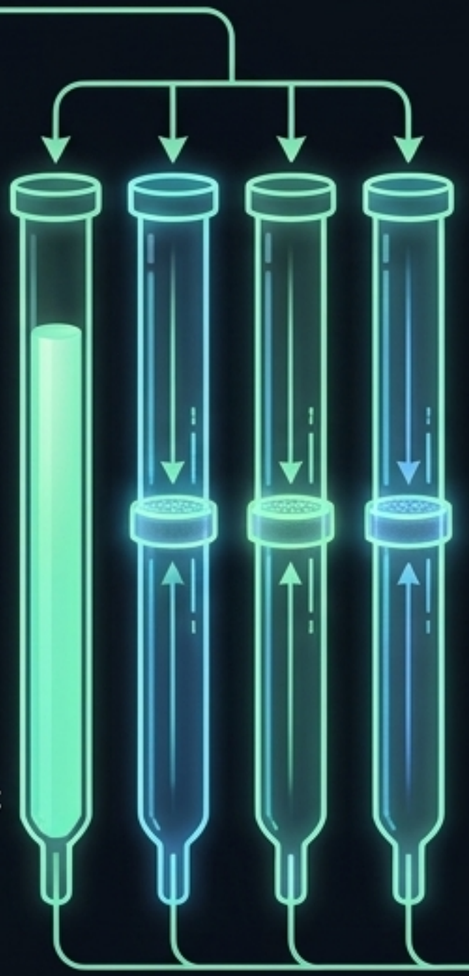
As context fills, latency scales linearly while instruction adherence degrades



Poisoning: Hallucinations or rejected turns get trusted trusted repeatedly.
Remedy: Rewind/Truncate. Do not compact.

Confusion: Bulky raw tool dumps dilute attention.
Remedy: Prune & Isolate (Pagination, Tool Search).

Clash: Contradictory rules accumulate.
Remedy: Scope path-specific rules via frontmatter.



Context Engineering Strategies

1. Write
(Externalize State)

2. Select
(Progressive Disclosure)

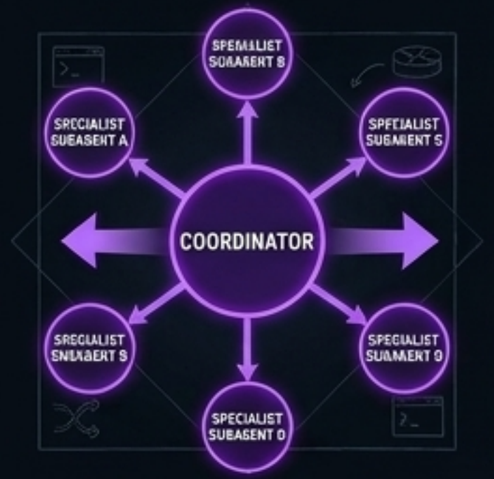
3. Compress
(Sliding Window Compaction)

4. Isolate
(Subagent Sandboxing)

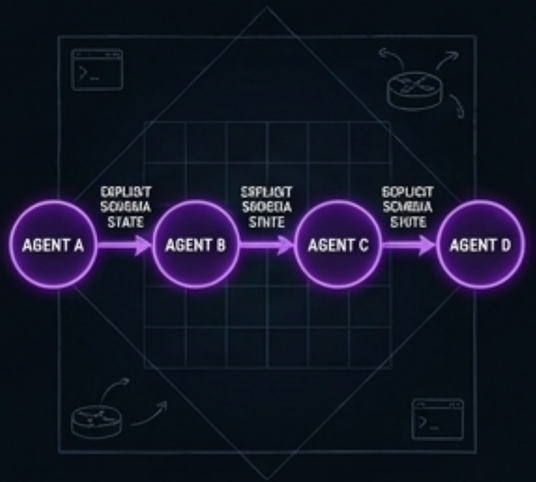
Multi-Agent Justification & Topologies

Beware "Agent Theater"—chaining identical persona prompts without context isolation only multiplies latency and drift.

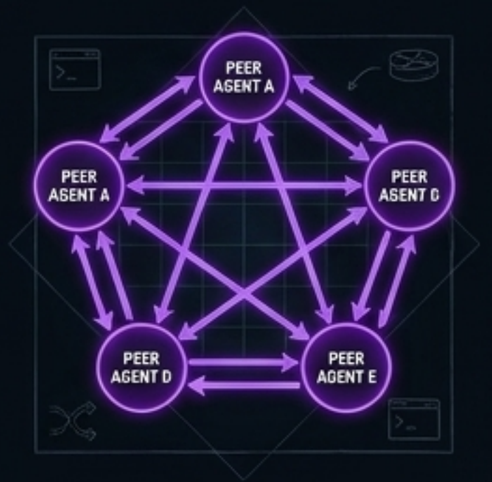
```
Multi-Agent Justified  ⟺  Distinct Tools OR Isolated Context OR Parallel I/O OR Adversarial Verification
```



1. Hub-and-Spoke
Central coordinator invokes specialist subagents. **Risk:** Hub bottleneck if spokes return massive raw dumps.



2. Sequential Pipeline
Fixed relay passing state via explicit schemas. **Benefit:** Zero routing overhead.



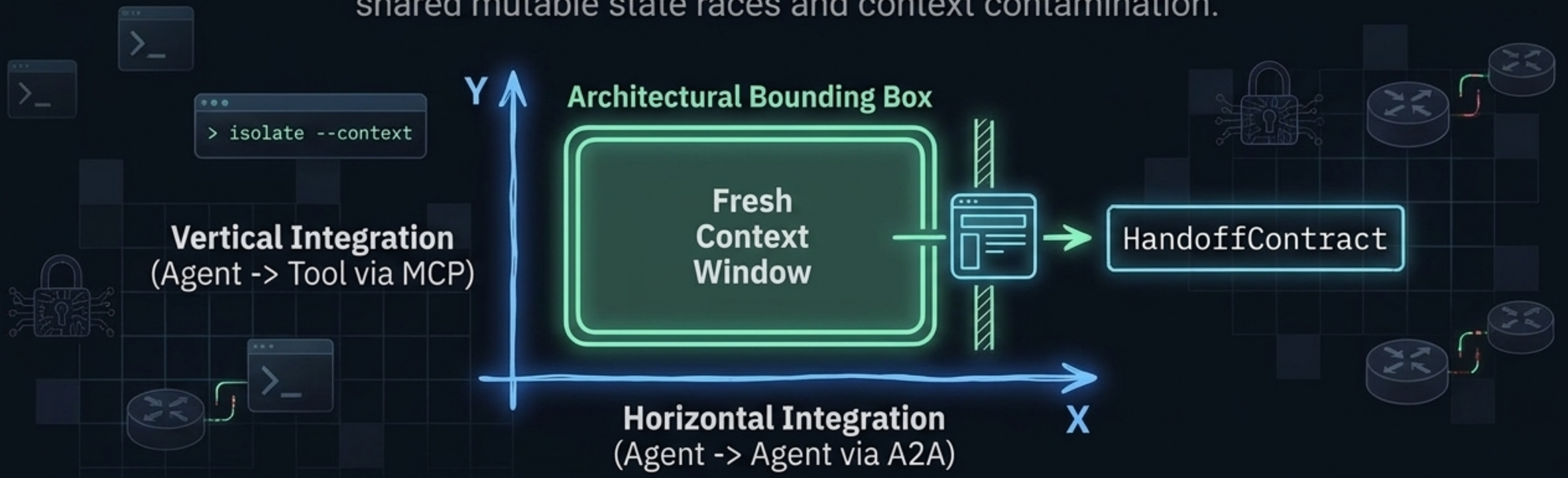
3. Mesh / Swarm
Peer specialists critique each other. **Risk:** Infinite ping-pong loops; requires deterministic tie-breakers.



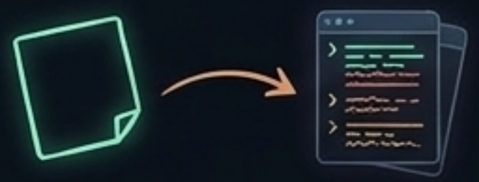
4. Hierarchical Teams
Multi-tier tree for massive workflows, keeping each node's tool catalog <= 16

The Context Isolation Rule & Handoff Contracts

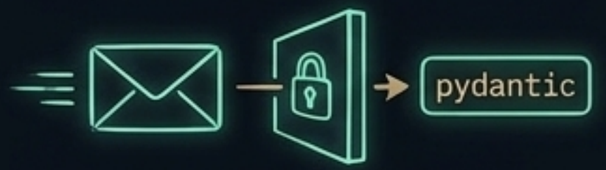
The single most important rule in multi-agent systems: preventing shared mutable state races and context contamination.



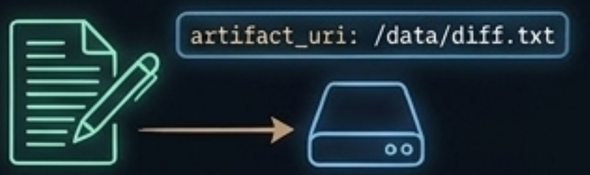
1. Fresh Context by Default: Subagents must start with a clean window containing only specialist instructions and task payload —not the coordinator's noisy history.



2. Handoff Contracts: Because intermediate tool calls stay private, only the final response crosses the boundary. Enforce typed `Pydantic` schemas.



3. Artifact Pointers: Never stuff a 2,000-line diff into the parent history. Write to disk and pass an `artifact_uri` pointer.



Production Anti-Patterns Diagnostic Matrix.

Troubleshooting Console

Symptom (Failure Mode)	Root Cause	Architectural Fix
Agent miscalculates fees/dates on edge cases.	Deterministic Drift	Move math to pre/post tool hooks (Python/SQL).
3-step pipeline takes 45s and costs $0.80.	Over-agentification (using ReAct where path is known)	Downgrade to Prompt Chain with fast-tier models.
Coordinator hits context limit after 4 workers.	Missing Context Isolation	Workers must return bounded HandoffContracts and write raw data to disk.
Generator-Critic loop burns 15 turns and still hallucinates.	Ungrounded reflection drift	Cap max_iterations=3 and verify citations deterministically via regex.

The 8-Point Architectural Checklist.

System launch criteria for deterministic hybrid intelligence.

- [x] **1. 3-Bucket Triage Verified:** Math/RBAC in code; high-stakes actions gated by HITL.
- [x] **2. Complexity Ladder Justified:** Empirical baseline proves simple workflows fail before deploying agents.
- [x] **3. Epistemic Gating Active:** Pre-LLM checks block corrupted/unreadable inputs for $0.
- [x] **4. Loop Hygiene Enforced:** max_turns, token ceilings, and duplicate tool-call interception active.
- [x] **5. ACI Poka-Yoke Hardened:** Catalogs <= 16 tools; constrained schemas; `is_error: true` structured signals.
- [x] **6. Context Budget Optimized:** Static rules pinned for cache; noisy exploration sandboxed in subagents.
- [x] **7. Handoff Contracts Typed:** Subagents run in fresh contexts and return strict JSON + artifact URIs.
- [x] **8. Separation of Authority:** High-stakes outputs verified by independent cold-context critic + code checks.

